ENTERPRISE ARCHITECTURE GUIDE | VERSION 1.0.0 | PRODUCTION READY

MeterFlow Systems

Usage-Based Pricing & Metering Engine

Comprehensive Architectural Guide, End-to-End Ingestion Flow, Sliding Window Aggregations, Complex Tier Pricing Algorithms, and Cloud Deployment Runbook.

> **System Mission:** High-throughput API event listener that calculates complex multi-tenant consumption metrics in real-time and feeds directly into enterprise billing systems (Stripe Billing Engine and Oracle NetSuite ERP).

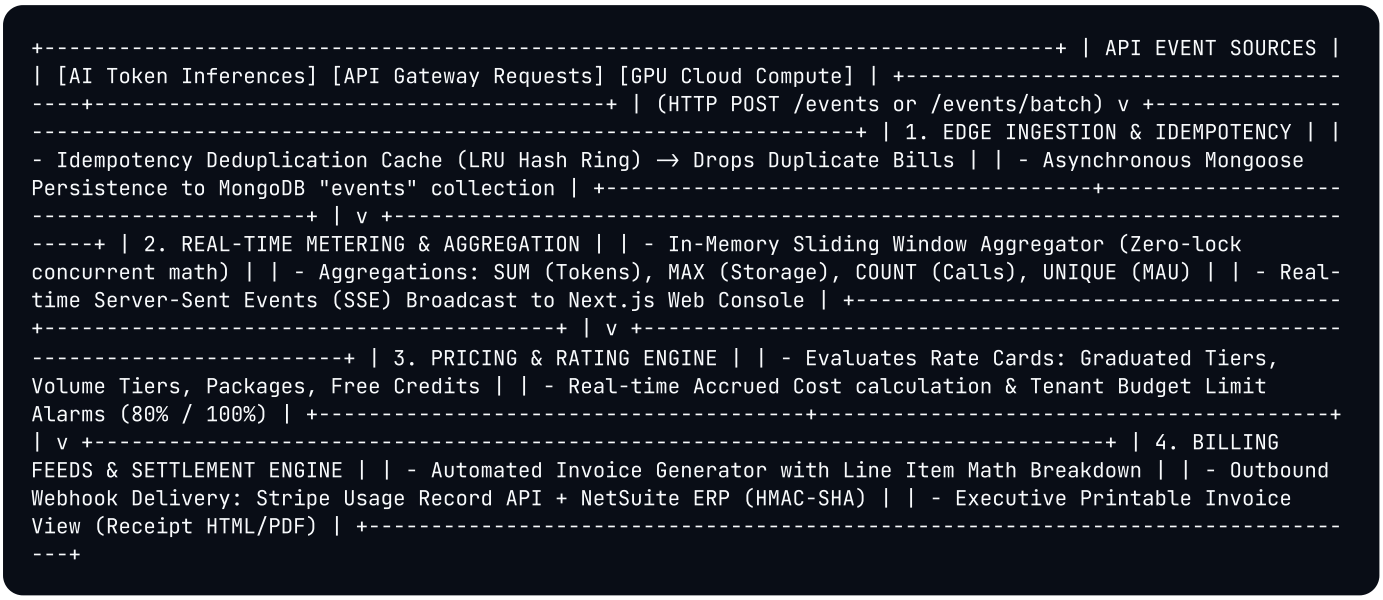
FRONTEND FRAMEWORK **Next.js 16 (App Router, Turbopack, Vanilla CSS)**	BACKEND ARCHITECTURE **Node.js + Express.js (ESM "type": "module", MVC)**
NOSQL DATABASE **MongoDB Atlas via Mongoose ORM**	REAL-TIME TRANSPORT **Server-Sent Events (SSE) Pipeline**

MeterFlow Cloud Infrastructure © 2026

Confidential - For Internal & Engineering Use

1. Architecture & End-to-End Workflow

The MeterFlow platform decouples raw event telemetry from financial calculation and billing systems. It operates in four coordinated phases: **Ingestion** → **Metering** → **Rating** → **Settlement**.



Core Architecture Components

⚡ Edge Ingestion Listener

Accepts single and batch telemetry payloads (up to 2,000/batch) with sub-millisecond overhead. Idempotency keys prevent double billing on network retries.

⏱️ Metering Aggregator

Maintains running totals per customer and meter. Supports mathematical formulas with filter expressions (e.g. `model == 'claude-3-5-sonnet'`).

🧮 Pricing Engine

Calculates consumption bills using multi-tier rate cards. Subtracts free allowances and computes graduated tier brackets with exact currency precision.

📦 Outbound Billing Feeds

Generates finalized invoices and delivers signed HMAC-SHA256 payloads to Stripe Billing API and corporate ERP accounting systems.

2. Consumption Meters & Aggregation Rules

A **Meter** defines the exact logic of how raw API telemetry translates into billable usage units. MeterFlow supports 6 mathematical aggregation functions:

AGGREGATION TYPE	MATHEMATICAL LOGIC	TYPICAL USE CASE	EXAMPLE PROPERTIES
SUM	∑(properties[valueProperty])	LLM Input/Output Tokens, Cloud Runtime Hours	`tokens: 2100`, `vCpuHours: 2.5`
MAX	Max(properties[valueProperty])	Peak High-Watermark Storage, Peak Concurrent DB Connections	`storageGb: 450.2`, `connections: 84`
COUNT	Count(matching events)	API Invocations, OCR Scans, Webhook Deliveries	`units: 1`, `endpoint: "/v1/inference"`
UNIQUE_COUNT	Cardinality of Set(properties[valueProperty])	Monthly Active Users (MAU), Unique Devices, IP Addresses	`userId: "usr_9410"`, `ip: "192.168.1.1"`
WEIGHTED_AVG	(∑ val × weight) / ∑ weight	Average Latency, Server Cluster Load	`duration: 120ms`, `weight: 5`
LATEST	Last recorded state value	Current Provisioned License Seats, Active Workers	`seatCount: 25`

Pre-configured Enterprise Meters

1. LLM Generative Tokens (`llm_tokens_total`)

```
SUM(ai.completion.properties.tokens)
```

Tracks token consumption across prompt and completion inferences.

2. API Gateway Invocations (`api_requests_total`)

```
COUNT(api.request.properties.units)
```

Counts billable HTTP and gRPC gateway requests.

3. GPU & Cloud Compute Hours (`gpu_compute_hours`)

```
SUM(cloud.compute.properties.vCpuHours)
```

Dedicated GPU clusters (NVIDIA A100) and vCPU compute runtime.

4. Monthly Active Users (`monthly_active_users`)

```
UNIQUE_COUNT(user.session.properties.userId)
```

Deduplicated unique user identities active within the billing cycle.

> **Property Extraction & Filters:** Meters support arbitrary filtering criteria. For example, a meter can track only events where `properties.model === "claude-3-5-sonnet"` or `properties.region === "us-east-1"`.

3. Complex Pricing Models & Rate Cards

MeterFlow provides mathematically rigorous pricing engines supporting four distinct billing models with included free credit allowances.

1. Graduated Tiered Pricing (Marginal Brackets)

Usage is partitioned into tiers. Each tier applies its unit rate only to the consumption falling within its specific range.

```
Example Rate Card: LLM Generative Tokens (Free Allowance: 100,000 tokens) Tier 1: 0 - 500,000 tokens @ $0.0020 / token Tier 2: 500,001 - 2,500,000 tokens @ $0.0015 / token Tier 3: 2,500,001+ @ $0.0008 / token If Consumed Quantity = 1,200,000 tokens: 1. Free Allowance: 100,000 tokens included ($0.00) 2. Billable Units: 1,200,000 - 100,000 = 1,100,000 billable units 3. Tier 1 Brackets: First 500,000 @ $0.0020 = $1,000.00 4. Tier 2 Brackets: Remaining 600,000 @ $0.0015 = $900.00 ---------- Total Accrued Cost = $1,000.00 + $900.00 = $1,900.00
```

2. Comparison of Pricing Models

MODEL	BEHAVIOR	FORMULA
GRADUATED	Marginal pricing per tier bracket	`Cost = ∑ (Units in Tier_i × Rate_i)`
VOLUME	Entire volume priced at single rate reached	`Cost = Total Units × Tier_Matched_Rate`
PACKAGE	Billed per block of units (rounded up)	`Cost = ⌈Total Units / PackageSize⌉ × PackagePrice`
FLAT_FEE	Flat pay-as-you-go rate per unit	`Cost = Total Units × UnitPrice`

3. Pre-configured Enterprise Plans

Enterprise AI & Scale ($499/mo Base)

Designed for high-scale generative AI applications. Graduated tokens (100k free, then $0.002/$0.0015/$0.0008), API package ($5 per 10k calls), GPU compute ($0.25/hr).

FinTech High-Throughput ($999/mo Base)

Volume discount pricing for financial transaction gateways. Aggressive volume tiers for API requests ($0.0008 down to $0.0002) with 100k free requests.

4. Billing Feeds & Webhook Integrations

Once consumption metrics are calculated, MeterFlow synchronizes directly with upstream financial accounting and payment engines via authenticated webhook pipelines.

Stripe Billing Usage Record Integration

Formatted according to the **Stripe Billing Meter Events API** specification, signed with `HMAC-SHA256` to ensure zero tampering:


```
{
  "id": "evt_hook_1730000000",
  "object": "event",
  "type": "billing.usage_record.created",
  "data": {
    "object": {
      "id": "mrec_inv_bb4b3211",
      "customer": "cust_anthropic_res",
      "subscription_item": "si_plan_enterprise_ai",
      "livemode": true,
      "quantity": 1100000,
      "total_amount_cents": 190000,
      "currency": "usd",
      "line_item_breakdown": [
        {
          "meter": "LLM Generative Tokens",
          "tier_details": [
            { "tier": 1, "unitsConsumed": 500000, "unitPrice": 0.002, "cost": 1000 },
            { "tier": 2, "unitsConsumed": 600000, "unitPrice": 0.0015, "cost": 900 }
          ],
          "amount": 1900.00
        }
      ]
    }
  }
}
```


Automated Invoice Generation & Printable HTML

Finalized invoices generate an itemized legal receipt accessible at:
`GET /api/v1/billing/invoices/:id/html`

> **Budget Limit & Threshold Alarms:** Each tenant account has a `budgetLimit` (e.g. $10,000). When accrued usage crosses **80%**, the dashboard displays amber telemetry warnings; at **100%**, automated soft or hard quota policies can be triggered.

5. Deployment Guide & API Reference

Production Cloud Deployment Runbook

Backend on Render / Railway

Build Command: `npm install`
Start Command: `node server.js`
Root Directory: `backend`
Environment Variables:

- `PORT=5001`
- `MONGO_URI=mongodb+srv://...`

Frontend on Vercel / Render

Build Command: `npm run build`
Output Directory: `.next`
Root Directory: `frontend`
Environment Variables:

- `NEXT_PUBLIC_API_URL=https://<backend>.onrender.com/api/v1`

API Reference Summary

ENDPOINT	METHOD	DESCRIPTION
`/api/v1/health`	GET	System uptime, memory, throughput rate, MongoDB connection status
`/api/v1/events`	POST	Ingest single billable API event with idempotency check
`/api/v1/events/batch`	POST	High-throughput batch ingestion (up to 2,000 events)
`/api/v1/events/stream`	GET	Real-time Server-Sent Events (SSE) telemetry pipeline
`/api/v1/meters`	GET / POST	List and define consumption meters and aggregation rules
`/api/v1/pricing-plans`	GET / POST	List and configure rate cards and graduated pricing tiers
`/api/v1/pricing-plans/simulate`	POST	Simulate tier calculation for arbitrary consumed units
`/api/v1/customers/:id/usage`	GET	Fetch live real-time accrued usage and budget status
`/api/v1/billing/invoices/generate`	POST	Finalize invoice & push webhook feed into Stripe/NetSuite
`/api/v1/simulator/burst`	POST	Trigger traffic burst (50, 150, 300 events) for load testing